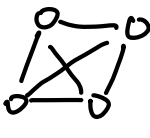
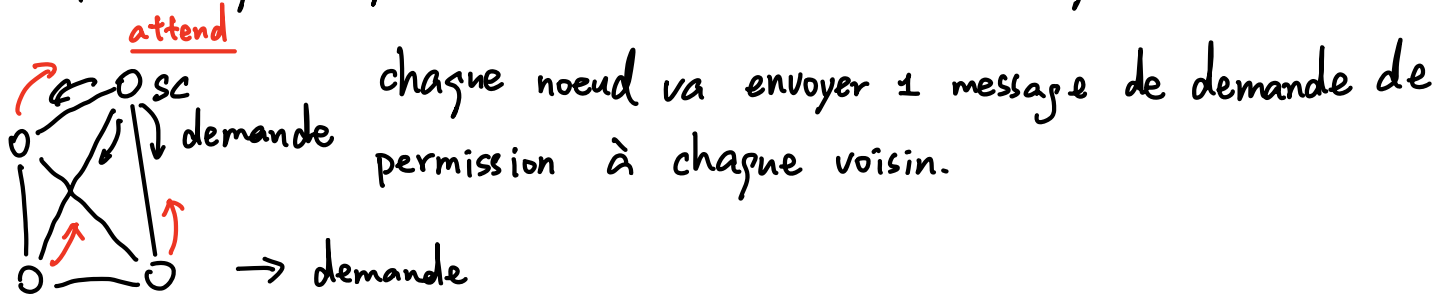


Exclusion mutuelle

Algo 3 : un réseau graphe complet 

Canaux : asynchrone, fiable.

Principe : chaque requête d'entrée en SC (section critique)



SC : le noeud est en SC
 S : en dehors de la section critique
 E : le noeud veut la section critique et attend.

demande SC : Envoyer $Dem(last_i, i)$ à $attendu_i$ ← date de dernier demande d'entrée en SC

Réception de Perm de j :
→ MAJ $attendu_i$

Si i est en SC
 $\Rightarrow$ Vrai
priorité ← $(etat_i = SC) \rightarrow$ la perm n'est pas envoyé.

$(etat_i = E) \wedge ((etat_i, i) < (h', j))$
 i en demande de SC
 j
 i et j en "concurrency" pour la SC

Algorithm 3 Algorithme de Ricart Agrawala, local au site i

Variables

$etat_i \in \{E, SC, S\}$. Initialisé à S .

h_i : entier initialisé à 0. C'est la dernière date connue de i et provenant des autres sites.

$last_i$: entier initialisé à 0. C'est la date de la dernière demande d'entrée en SC de i .

$differe_i$: ens. des sites dont l'envoi de la permission est retardée. Initialisé à $\emptyset$.

$attendu_i$: ens. des sites dont i attend la permission. Initialisé à $\emptyset$.

$priorite_i$: booléen qui est à vrai si i est prioritaire par rapport à la demande en cours de traitement. Initialisé à faux.

— Algorithmme —

Sur demande d'entrée en section critique $\rightarrow$

```

etati  $\leftarrow$  E
lasti  $\leftarrow$  hi + 1
attendui  $\leftarrow$  tous les sites sauf i
for all j  $\in$  attendui do
    Envoyer Dem(lasti, i) à j
    
```

Sur réception de Dem(h', j) de j $\rightarrow$

```

hi  $\leftarrow$  max(hi, h')
prioritei  $\leftarrow$  (etati = SC)  $\vee$  [(etati = E)  $\wedge$  (lasti, i) < (h', j)]
if prioritei = vrai then
    differei  $\leftarrow$  differei  $\cup$  {j}
else
    Envoyer Perm à j
    
```

Sur réception de Perm de j $\rightarrow$

```

attendui  $\leftarrow$  attendui  $\setminus$  {j}
if attendui =  $\emptyset$  then
    etati  $\leftarrow$  SC
    
```

Sur sortie de section critique $\rightarrow$

```

etati  $\leftarrow$  S
for all j  $\in$  differei do
    Envoyer Perm à j
differei  $\leftarrow$   $\emptyset$ 
    
```

Partie 1 : $\uparrow$ exclusion mutuelle avec demande / permission.

Partie 2 : exclusion mutuelle avec circulation jeton. : ([]) $\leftarrow$ tableau avec autant de cases que de sites.

1 algo Lelam jeton.

l'hypothèse : un anneau orienté

Connaissance : succ_i

Variable : { S, SC, E }

Algorithme 4

Sur demande d'entrée en SC :

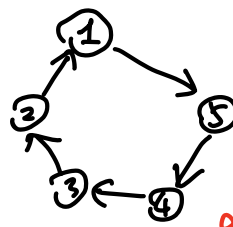
Etat_i $\leftarrow$ E

Sur Réception de Jeton :

Si Etat = E

 | Etat = SC

Sinon envoyer Jeton() à succ_i



$\rightarrow$ la tableau contient le nombre de demandes effectuées par ce site lorsque le jeton était sur ce site par la dernière fois

nbDem_i[] : tableau qui contient Le nombre des demande d'entrée en SC des différents sites.

entré en SC : nbDem_i[i] envoyer Dem()

for all j = i+1 à n puis 1 à i-1

 | if Jeton[j] < nbDem[j] n Jeton Present_i

Sur Sorti de SC

Etat $\leftarrow S$

Envoyer Jeton() à succi.

┌ JetonPresenti $\leftarrow$ faux
└ Envoyer Jeton() à j.

3 Algos d'élection

topologie : anneau orienté identifié.

Le Lann 1977

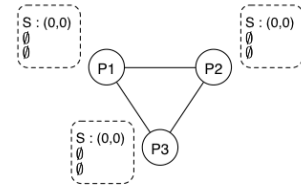
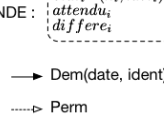
Algo 1: certains sites se déclarent candidats, ils envoient leur id dans l'anneau.

Tous les messages font le tour de l'anneau.

Un site candidat conserve tous les id. Lorsque un candidat reçoit son propre id, il se déclare leader s'il a le plus petit id parmi tous les candidats

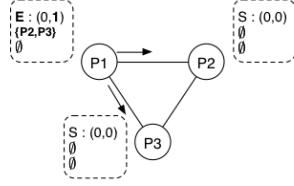
Exécution Simple : une seule demande

LEGENDE :



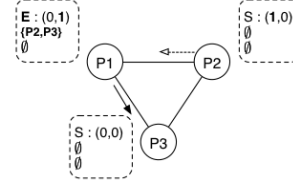
1) P1 demande à entrer en SC

Met à jour son état (E) et la date de sa dernière demande (1)
Envoie Dem(1,P1) à tous
attendu := {P2,P3}



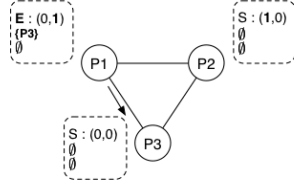
2) P2 reçoit la demande de P1

Met à jour son compteur priorité := Faux
Envoie sa permission à P1



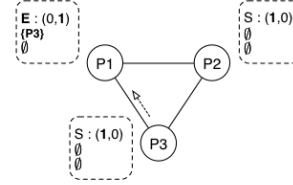
3) P1 reçoit la permission de P2

attendu := {P3}



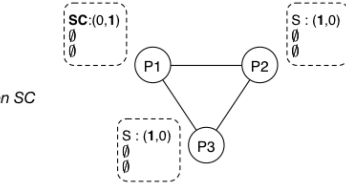
4) P3 reçoit la demande de P1

Met à jour son compteur priorité := Faux
Envoie sa permission à P1



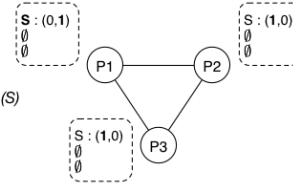
5) P1 reçoit la permission de P3

attendu := {}
P1 change son état (SC), puis entre en SC



6) P1 sort de SC

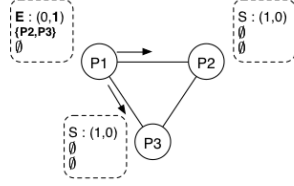
attendu := {}
P1 change son état (S)



La suite : deux processus demandent quasi en même temps (à partir de la configuration en (6)).

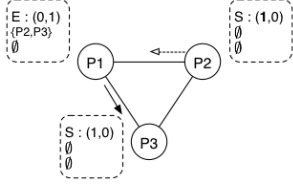
7) P1 demande à entrer en SC

Met à jour son état (E) et La date de sa dernière demande reste inchangée !!
Envoie Dem(1,P1) à tous
attendu := {P2,P3}



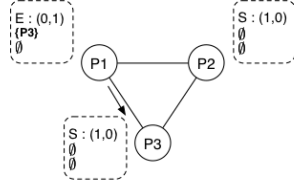
8) P2 reçoit la demande de P1

Met à jour son compteur priorité := Faux
Envoie sa permission à P1



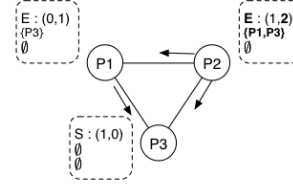
9) P1 reçoit la permission de P2

attendu := {P3}



10) P2 demande à entrer en SC

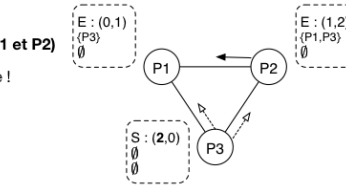
Met à jour son état (E) et la date de sa dernière demande (2)
Envoie Dem(2,P2) à tous
attendu := {P1,P3}



11) P3 reçoit les deux demandes (de P1 et P2)

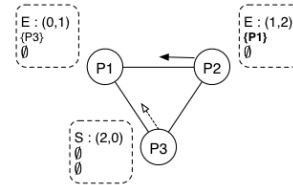
ATTENTION : 2 transitions en une !

Met à jour son compteur (2)
priorité := Faux
Envoie sa permission à P1 et à P2



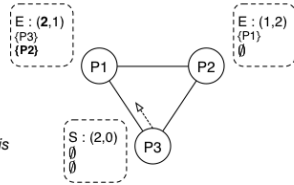
12) P2 reçoit la permission de P3

attendu := {P1}



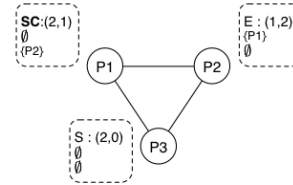
13) P1 reçoit la demande Dem(2,P2) de P2 (différé !)

Met à jour son compteur (2)
priorité := Vrai
N'envoie pas sa permission à P2, mais met P2 dans différé



14) P1 reçoit la permission de P3

attendu := {}
P1 change son état (SC), puis entre en SC



15) P1 sort de SC

P1 change son état (S)
Envoie sa permission à P2
Vide différé

